

UNIVERZITET U TUZLI
FAKULTET ELEKTROTEHNIKE
TELEKOMUNIKACIJE I INFORMACIONE TEHNOLOGIJE



Predmet: Infrastruktura i servisi u oblaku

Projektna Dokumentacija

**Notes Aplikacija
Kontejnerizovana aplikacija na AWS-u**

Kandidat:

Abdulhalim Šestan
Indeks: 22159

Predmetni nastavnici:

Prof. dr. Alma Šećerbegović
Asist. Lejla Šarić

Tuzla, maj 2026. godine

Sadržaj

1	Uvod i Pregled Sistema	2
2	Arhitektura Sistema (Kako smo ovo povezali)	2
2.1	Slojevi aplikacije	2
2.2	Mrežna topologija i objašnjenje podsistema	2
2.3	Kako radi Load Balancer?	2
2.4	Sigurnosne grupe (Security Groups)	3
3	Finansijski Izvještaj	3
4	Kako Pokrenuti Sve Ovo (Uputstvo za deployment)	4
4.1	Pokretanje preko Terraform-a	4
4.2	Šta Terraform radi	4
4.3	Opcija B: Ručni deployment (PowerShell)	5
4.4	Opcija C: Lokalno testiranje (Docker)	5
4.5	Kako pristupiti aplikaciji nakon deployment-a	5
4.6	Kako očistiti AWS resurse nakon testiranja	5
5	Izazovi u projektu	6
6	Čišćenje Resursa i Zaključak	6
7	Dijagram	7

1. Uvod i Pregled Sistema

U okviru ovog projekta imali smo zadatak da napravimo i pokrenemo modernu, troslojnu web aplikaciju, u mom slučaju aplikaciju za bilješke. Cilj je bio da aplikacija ne bude samo "kod koji radi na mom računaru", nego da je dignemo na AWS cloud (nastavak od prvog projekta) tako da bude sigurna, skalabilna i visoko dostupna (*High Availability*).

Da ne bismo sve kliknuli ručno kroz AWS konzolu, koristili smo **Terraform**. To nam omogućava da svu infrastrukturu opišemo kroz kod što je bio drugi zadatak projekta. Kad nam treba sistem, opalimo jednu komandu i sve se stvori za 10 minuta; kad završimo sa vježbama, obrišemo sve jednom komandom da ne trošimo lab kredite.

2. Arhitektura Sistema (Kako smo ovo povezali)

Aplikaciju smo podijelili na tri glavna dijela (sloja) kako bi sve radilo nezavisno i stabilno:

2.1. Slojevi aplikacije

Sloj	Tehnologija / Servis	Objašnjenje (Šta zapravo radi)
Frontend	S3 Static Website	Ovdje stoje obični <code>index.html</code> , <code>style.css</code> i <code>script.js</code> . S3 ovdje glumi web server. Pošto je konfigurisan kao <i>Static Website</i> , on samo servira te fajlove korisniku u browser i ništa više.
Backend	EC2 + Docker + Flask	Naš Flask API koji žvaće podatke. Da osiguramo stabilnost, digli smo dvije iste EC2 instance (<code>t2.micro</code>) u dvije različite Availability Zone (<i>us-east-1a</i> i <i>us-east-1b</i>). Unutar njih se vrti Docker kontejner sa Flaskom.
Baza	RDS MySQL 8.0	Prava SQL baza podataka (<code>db.t3.micro</code>) sa 20GB prostora. Nismo je instalirali ručno na EC2, nego smo uzeli AWS-ov upravljani RDS servis jer se lakše održava.

Tablica 1: Pregled slojeva aplikacije

2.2. Mrežna topologija i objašnjenje podsistema

Cijeli sistem smo zatvorili u naš privatni oblak (VPC) sa mrežnim opsegom `10.0.0.0/16`. Tu smo morali napraviti podjele na podmreže (subnete) kako bismo osigurali bazu:

- **Public Subnets** - Tu smo stavili **Application Load Balancer** i **EC2 instance**. Oni imaju dodijeljen Internet Gateway i mogu da pričaju sa vanjskim svijetom.
- **Private Subnets** - Tu je sakrivena **RDS MySQL baza**. Baza nema javnu IP adresu i niko s interneta ne može direktno da joj pristupi, što je odlično za sigurnost. Kad bazi zatreba izlaz na internet (npr. za neki update), ona ide preko **NAT Gateway-a** koji radi samo u jednom smjeru.

2.3. Kako radi Load Balancer?

Pošto imamo dvije EC2 instance, potreban nam je neki segment koji će dijeliti posao među njima. Tu uskače ALB koji radi takozvano *Path-Based* rutiranje:

1. Ako u URL-u vidi `/api/*`, on skonta: "Aha, ovo je za bazu i podatke" i šalje zahtjev na naše Flask EC2 instance u Target Grupu.
2. Za sve ostale zahtjeve, ALB radi **HTTP 301 Redirect** i preusmjerava korisnika na S3 URL gdje se nalazi naš frontend.

2.4. Sigurnosne grupe (Security Groups)

Ovdje smo primijenili neki fazon što je u biti taktika da ne vjerujemo nikome:

- `notes-alb-sg`: Otvoren prema svima (0.0.0.0/0) ali samo na portu 80.
- `notes-backend-sg`: Pušta saobraćaj na port 5000 **samo i isključivo** ako on dolazi sa Load Balancer-a (`notes-alb-sg`). Otvorili smo i port 22 (SSH) za debugovanje s našeg računara.
- `notes-rds-sg`: Pušta saobraćaj na port 3306 (MySQL) **samo ako dolazi sa backend instanci**. Znači, bazu niko ne može ni "pingati" osim samog backenda.

3. Finansijski Izvještaj

Cijene su izvučene iz *AWS Pricing Calculator*-a za regiju `us-east-1`. Srećom, u sklopu **AWS Academy Learner Lab-a** imamo budžet koji ovo pokriva, pa nas je sve koštalo **0\$**.

Mjesečni troškovi	
• EC2 Računanje: 2 komada <code>t2.micro</code>	\$16.94
• RDS Baza podataka: <code>db.t3.micro</code> + 20GB prostora	\$14.71
• Application Load Balancer: 1 komad	\$22.27
• NAT Gateway: 1 komad (Najskuplja stavka)	\$32.85
• S3 i mrežni promet: Par fajlova i oko 5GB protoka	\$0.45
• Ukupni mjesečni trošak:	~\$87.22

2.2 Uštede (ako se isključi kada se ne koristi)

Strategija	Opis	Mjesečni trošak
24/7	Sve radi stalno	~\$87
Radno vrijeme	8h/dan × 22 dana	~\$21
Samo testiranje	Ugasiti kad se ne koristi	\$0–5

2.3 Procjena godišnjeg troška

- **Produkcioni scenario (24/7):** ~\$1,044 godišnje
- **Razvojni scenario (8h × 22 dana):** ~\$252 godišnje
- **AWS Academy (lab budget):** \$0 (pokriveno lab kreditima)

Servis	Cijena	Detalji
t2.micro	\$0.0116/h	2 vCPU, 1 GB RAM
db.t3.micro	\$0.017/h	2 vCPU, 1 GB RAM, MySQL
EBS gp2 (RDS)	\$0.115/GB/mj	20 GB alocirano = \$2.30/mj
S3 Standard	\$0.023/GB/mj	Za ~50KB frontend: zanemarivo
ALB	\$0.0225/h + \$0.008/LCU-h	1 LCU = 25 novih veza/s, 1000 aktivnih zahtjeva/s
NAT Gateway	\$0.045/h + \$0.045/GB	Obrada podataka kroz NAT
Data transfer out	\$0.09/GB (prvih 10 TB)	Do interneta sa EC2/ALB

4. Kako Pokrenuti Sve Ovo (Uputstvo za deployment)

Morate imati instaliran Terraform, AWS CLI, Docker i Git.

4.1. Pokretanje preko Terraform-a

```
# 1. Klonirajte nas projekat
git clone https://github.com/a-sestan/notes-app.git
cd notes-app/terraform

# 2. Unesite privremene kljuceve sa AWS Academy
aws configure set aws_access_key_id "ASIA..."
aws configure set aws_secret_access_key "CNYS..."
aws configure set aws_session_token "IQoJ..."
aws configure set region "us-east-1"

# 3. Pripremite varijable
cp terraform.tfvars.example terraform.tfvars

# 4. Pokrenite
terraform init
terraform plan
terraform apply
```

Listing 1: Komande za pokretanje projekta

Sada možemo otići popiti kafu u Merkatoru jer bazi (RDS) treba oko 10 minuta da se podigne na AWS-u. Kad završi, terminal će ispisati URL adrese preko kojih pristupate aplikaciji.

4.2. Šta Terraform radi

- Kreira VPC (10.0.0.0/16) sa 2 public i 2 private subnet-a
- Kreira Internet Gateway i NAT Gateway
- Kreira 3 Security Groups sa pravilima
- Kreira RDS MySQL bazu (db.t3.micro)
- Kreira S3 bucket i uploaduje frontend fajlove
- Kreira 2 EC2 instance sa Docker-om i Flask backendom

- Kreira ALB sa Target Group, listenerom i path-based routing pravilom
- Automatski postavlja API_BASE u script.js na ALB DNS

4.3. Opcija B: Ručni deployment (PowerShell)

```
cd scripts
.\deploy-all.ps1 '
    -AccessKey "ASIA..." '
    -SecretKey "CNYS..." '
    -SessionToken "IQoJ..." '
    -KeyPrivatePath "C:\path\to\labsuser.pem"
```

Detaljna uputstva za ručni deployment potražite u README.md.

4.4. Opcija C: Lokalno testiranje (Docker)

```
cd notes-app

# Pokrenite MySQL + backend + frontend
docker-compose up -d

# Otvorite u browseru:
#   http://localhost:8080    (frontend)
#   http://localhost:5000    (backend API)
```

4.5. Kako pristupiti aplikaciji nakon deployment-a

URL	Namjena
<code>http://<s3-bucket>.s3-website-us-east-1.amazonaws.com</code>	Frontend — otvorite u browseru
<code>http://<alb-dns>/api/notes</code>	API — direktan pristup (JSON)
<code>http://<alb-dns>/api/notes (POST)</code>	Kreiranje bilješke

4.6. Kako očistiti AWS resurse nakon testiranja

Terraform:

```
cd terraform
terraform destroy -auto-approve
```

Ručno (ako ste koristili deploy skripte):

```
aws elbv2 delete-load-balancer --load-balancer-arn "$ALB_ARN"
aws elbv2 delete-target-group --target-group-arn "$TG_ARN"
aws ec2 terminate-instances --instance-ids "$INSTANCE_1" "$INSTANCE_2"
aws rds delete-db-instance --db-instance-identifier notes-db --skip-final-snapshot
aws s3 rm "s3://$BUCKET" --recursive
aws s3 rb "s3://$BUCKET"
aws ec2 delete-security-group --group-id "$RDS_SG"
aws ec2 delete-security-group --group-id "$BACK_SG"
aws ec2 delete-security-group --group-id "$ALB_SG"
aws ec2 delete-key-pair --key-name notes-app-key
```

5. Izazovi u projektu

Zašto nam se frontend na početku nije htio učitati? Kada smo prvi put podesili Load Balancer, stavili smo da sve ide na backend. Kada otvorimo stranicu, a browser izbaci grešku jer Flask backend nema pojma šta je `index.html`.

Rješenje je sledeće - Skontali smo da moramo napraviti pravila na ALB listeneru. Rekli smo mu: "Ako putanja sadrži `/api/`*, šalji na Flask. Za sve ostalo, radi redirect na S3 adresu".

Problem sa praznom varijablom `API_BASE` u JavaScriptu U fajlu `script.js` stoji linija `API_BASE = ''`. Tu frontend treba da zna tačnu adresu Load Balancer-a. Ali kvaka je što mi tu adresu ne znamo ipadrijeđ, jer je AWS generiše tek kad se napravi Load Balancer.

Rješenje je sledeće - Ovo smo riješili trikom unutar samog Terraform koda (`s3.tf`). Kada Terraform uploaduje `script.js` na S3 bucket, on iskoristi funkciju `replace()`. Ona automatski zamijeni prazan string sa pravom DNS adresom Load Balancer-a.

AWS Academy nam nije dao najnoviji Linux Pratili smo tutorijale s interneta koji koriste Amazon Linux 2023. Međutim, AWS Academy nam je stalno bacao greške tipa `RunInstances denied`. Ispostavilo se da studentski nalozi imaju restrikcije.

Rješenje je sledeće - Prebacili smo se na stariji, provjereni **Amazon Linux 2**.

Instanci nikad dosta zdravija - vazda su bolesne Load Balancer nam je uporno izbacivao da su EC2 instance `unhealthy` iako se Docker kontejner uredno pokrenuo unutar njih.

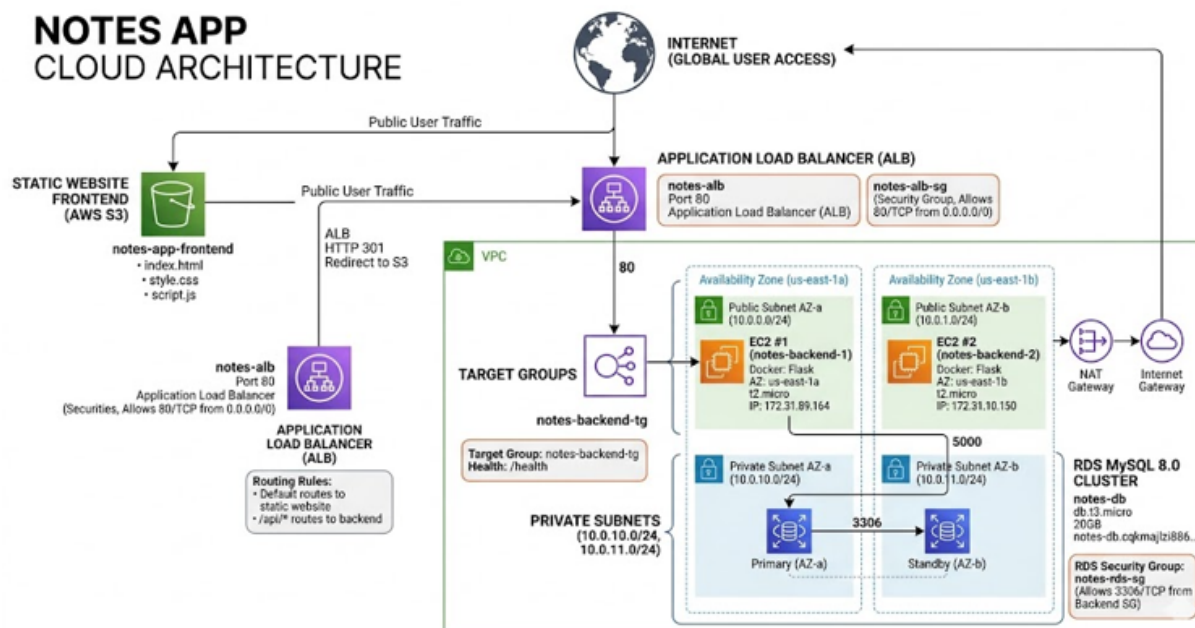
Rješenje je sledeće - Skontali smo da ALB traži health check putanju. Mi smo stavili `/health`, ali naš Flask uopšte nije imao definisanu tu rutu. Brže-bolje smo u `app.py` dodali jednostavnu rutu koja vraća HTTP 200, i status na konzoli je pozelenio.

6. Čišćenje Resursa i Zaključak

Iz direktorijuma `terraform` samo pokrenemo komandu `terraform destroy -auto-approve`. Terraform će sam, obrnutim redoslijedom, sve obrisati. Nakon toga kliknemo na **"End Lab"** u AWS Academy konzoli.

Kroz ovaj projekat smo iz prve ruke osjetili šta znači raditi u cloud okruženju i koliko sitnice u mrežama mogu da vam oduzmu vremena - previše a i živci su postali tanji. Na kraju, aplikacija uspješno radi, podaci se spašavaju u bazu, a arhitektura je napravljena po školskim pravilima.

7. Dijagram



Slika 1: Dijagram